TUGAS 1

Login dengan Password Terenkripsi

Laravel 13

Tujuan	Mahasiswa mampu membuat sistem login yang aman dengan Laravel, mencakup password ter-hash, proteksi halaman, dan logout.
Prasyarat	PHP 8.3 atau lebih tinggi, Composer, dan MySQL (XAMPP/Laragon). Laravel 13 dirilis pada 17 Maret 2026 dan membutuhkan PHP 8.3 atau lebih tinggi.

Tampilan Acuan (Mockup)

1. /login (contoh saat login gagal)

```
+--------------------------------------+
| localhost:8000/login                 |
+--------------------------------------+
|  Login                               |
|  Masuk untuk membuka dashboard       |
|                                      |
|  [ Username atau password salah. ]   |
|                                      |
|  Username                            |
|  [ budi                          ]   |
|  Password                            |
|  [ ********                      ]   |
|                                      |
|  [            Masuk              ]   |
+--------------------------------------+
```

2. /dashboard (setelah login berhasil)

```
+--------------------------------------+
| localhost:8000/dashboard             |
+--------------------------------------+
|  Dashboard                [ Logout ] |
|--------------------------------------|
|  Selamat datang, Budi Santoso!       |
|  Halaman ini hanya bisa dibuka       |
|  setelah login.                      |
|  Username: budi                      |
+--------------------------------------+
```

3. /dashboard dibuka tanpa login

```
localhost:8000/dashboard
        |
        |  pengguna belum login
v
otomatis dialihkan ke localhost:8000/login
```

Ketentuan

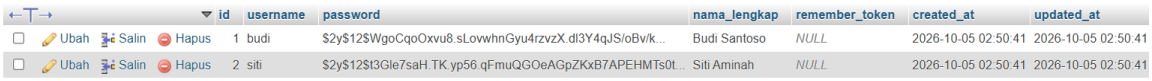
---

1. Buat proyek dengan `composer create-project laravel/laravel:^13.0 tugas1`. **Dilarang memakai starter kit** (Breeze, Jetstream, dan sejenisnya); logika login ditulis sendiri.
2. Atur `.env` agar memakai MySQL (`DB_CONNECTION=mysql`, `DB_DATABASE=db_tugas`, dan seterusnya). Buat database kosong `db_tugas`.
3. Tabel `users` dibuat lewat **migration** dengan kolom: `id`, `username` (unik), `password`, `nama_lengkap`, `timestamps`. Ubah migration bawaan agar memakai `username`, bukan `email`.
4. Isi minimal 2 user lewat **seeder** (`php artisan db:seed`). Password disimpan dalam bentuk hash. Pastikan model `User` menyesuaikan kolom baru (`$fillable`) dan memakai cast `'password' => 'hashed'`.
5. Buat `AuthController` dan route berikut:
   - `GET /login`: menampilkan form (route diberi nama `login`).
   - `POST /login`: memvalidasi input, lalu memakai `Auth::attempt()`.
     - Jika berhasil: `$request->session()->regenerate()` dan arahkan ke `/dashboard`.
     - Jika gagal: kembali ke form dengan pesan "Username atau password salah." tanpa menyebut mana yang salah.
   - `GET /dashboard`: menampilkan "Selamat datang, [nama_lengkap]!" dan tombol **Logout**.
   - `POST /logout`: memanggil `Auth::logout()`, `session()->invalidate()`, `session()->regenerateToken()`, lalu mengarahkan ke `/login`.
6. Proteksi route:
   - `/dashboard` dan `/logout` memakai middleware `auth`. Dashboard yang dibuka langsung lewat URL tanpa login otomatis dialihkan ke `/login`.
   - `/login` memakai middleware `guest`. Pengguna yang sudah login langsung diarahkan ke dashboard.
7. Tampilan memakai **Blade** (`login.blade.php`, `dashboard.blade.php`).

Pertanyaan (maksimal 1 halaman)

a. Mengapa password harus di-hash? Lihat kolom `password` di tabel `users` dan jelaskan isinya.

Jawaban : password harus di hash agar tidak bisa dibaca sembarangan oleh orang yang tidak bertanggung jawab, bahkan admin atau developer sekalipun tidak dapat langsung mengetahui password user tersebut, jadi ketika user mendaftarkan atau membuat password untuk pertamakalinya misal 'akuganteng123', maka 'akuganteng123' akan di hash menjadi huruf yang terlihat acak misal 'abcd12345efgh', nah hash ini akan disimpan di database dan sifatnya unik hanya untuk kata 'akuganteng123'. Jadi ketika user memasukan kembali password 'akuganteng123' otomatis hash nya cocok dengan 'abcd12345efgh'.



	id	username	password	nama_lengkap	remember_token	created_at	updated_at
Ubah Salin Hapus	1	budi	$2y$12$WgoCqoOxvu8.sLovwhnGyu4rzvzX.dl3Y4qJS/oBv/k...	Budi Santoso	NULL	2026-10-05 02:50:41	2026-10-05 02:50:41
Ubah Salin Hapus	2	siti	$2y$12$t3Gle7saH.TK.yp56.qFmuQGOeAGpZKxB7APEHMTs0t...	Siti Aminah	NULL	2026-10-05 02:50:41	2026-10-05 02:50:41

b. Mengapa logout memakai metode POST dengan `@csrf`, bukan link biasa?

Jawaban : logout menggunakan method post agar tidak dapat di akses lewat URL melainkan harus langsung lewat body atau struktur HTML misal tombol fungsinya jika ada orang yang tidak bertanggung jawab, dia bisa menglogout akun user lewat url, sedangkan @csrf saya masih beum terlalu paham tapi dari yang saya baca csrf ini digunakan untuk session dimana setiap session itu punya id tersendiri nah dengan ditambahkannya csrf ini maka setiap session punya id dan token csrf ini sehingga untuk masuk ke session diperlukan 2 penjaga yaitu id session dan toke csrf.

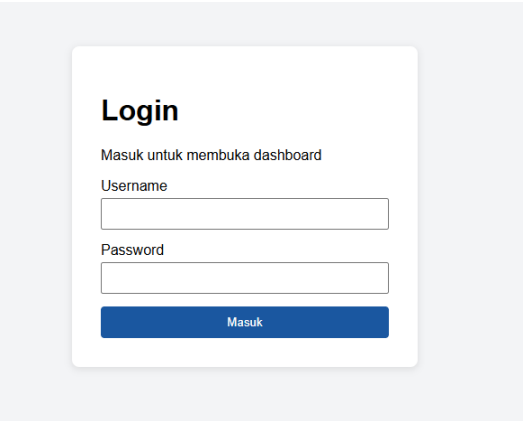
c. Apa fungsi middleware `auth` dan `guest`? Apa yang terjadi jika `/dashboard` dibuka tanpa login?

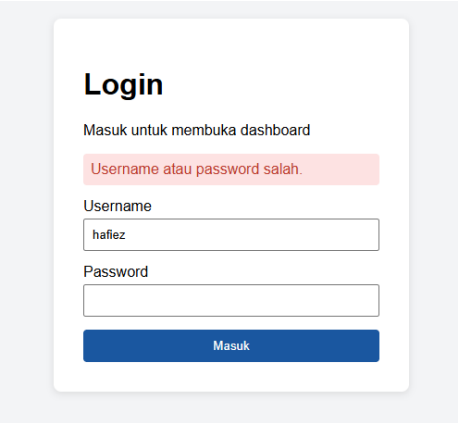
Jawaban : Berfungsi untuk melindungi halaman yang hanya boleh diakses oleh pengguna yang sudah login, sedangkan guest berfungsi untuk melindungi halaman yang hanya boleh diakses oleh pengguna yang BELUM login.

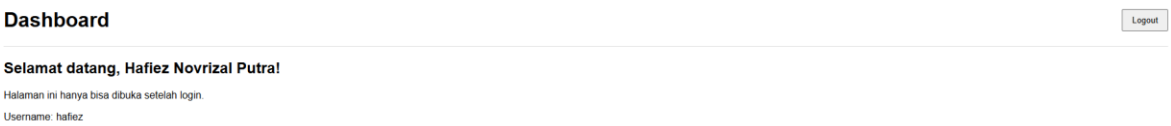
d. Apa fungsi `session()->regenerate()` setelah login berhasil?

Jawaban : session()->regenerate() membuat ID session baru setelah login dan membuang ID lama (data session tetap dipertahankan). Ini mencegah penyerang yang mengetahui ID session sebelum login lalu ikut memakai sesi yang sudah terautentikasi setelah korban login. Setelah ID diganti, ID lama yang dipegang penyerang tidak berlaku lagi.

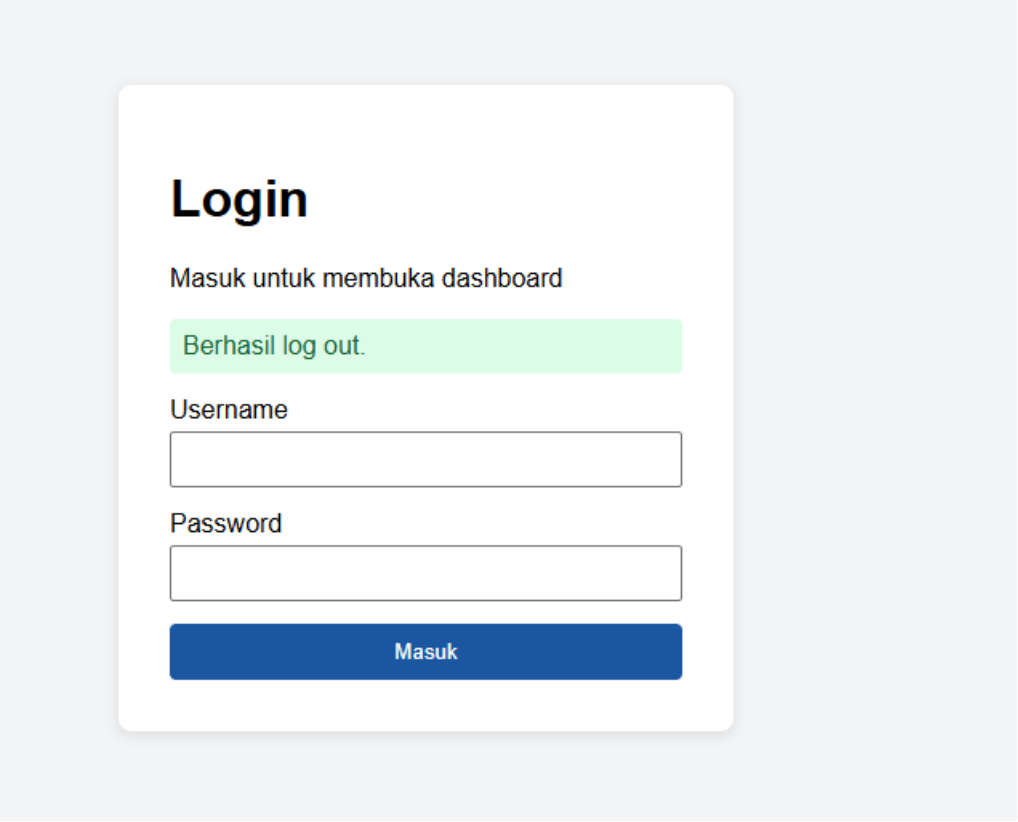
Screenshoot Tampilan







Tidak dapat masuk dengan /dashboard karena tidak ada id user di sessions yang auth sehingga di kembalikan ke menu /login.



Yang Dikumpulkan

- Folder proyek.
- Screenshot:
- isi tabel `users` (password berupa hash)
- login gagal
- login berhasil
- dashboard
- `/dashboard` dibuka di jendela incognito tanpa login
- setelah logout
- Jawaban pertanyaan.